A Policy-Bounded Agentic Framework for Autonomous Governance of Cloud Data Pipelines

B Sai Krishna, Vignes VM, I Mahalakshmi, and Sri Harini MP

Abstract—A recent proposal governs cloud data pipelines with four policy-gated LLM agents and reports substantial gains over static orchestration, without saying which model-in-the-loop condition produced them. We implement the system and evaluate it with a real, remote language model in a 560-run campaign that crosses four injected fault scenarios with eight configurations: three non-agent comparators, the full system, and four single-agent ablations. With a live model, the full system changes mean time to recovery (MTTR) by −38% and cost by −56% relative to static orchestration. An assumed provisioning gap accounts for most of the cost change. Most of the MTTR change traces to a flaw we found afterwards in the simulated on-call engineer: it drew one delay per escalation and kept the fastest. The full system also increases manual interventions by 500%. Two idealized non-LLM baselines with fixed modelled latencies recover faster still, changing MTTR by −92% and −86%. When the same full configuration runs under a deterministic mock instead of the live model, recovery time falls to 0.193 s, about three orders of magnitude below the live figure, and the correct-mitigation score rises from 0.42 to 1.00. A mock evaluation and a live evaluation of the same system therefore support different conclusions. Graded against an oracle independent of the policy code, a generated adversarial corpus of 5547 cases yields containment ≥ 0.9990, but only after two contract-layer gaps it exposed were fixed. The evidence is bounded by a single host, synthetic data with injected faults, a simulated on-call engineer, one live model family, and baselines that are modelled rather than deployed.

Index Terms—LLM agents, cloud data pipeline governance, policy-as-code, Open Policy Agent, AIOps, evaluation methodology, adversarial evaluation, mock-vs-live evaluation validity

I. INTRODUCTION

In practice, the failures that stop a data pipeline tend to be ordinary ones: a column changes type upstream, a source goes quiet, a co-tenant takes the compute the pipeline needs, or a burst of ingress exceeds capacity. Foidl et al. counted 41 separate factors behind quality and reliability failures in real pipelines [1]. The usual fix is still a person on call, and much of the recovery time is simply the time that person needs to respond. Large language model (LLM) agents could cut this loop short. Several agent systems for cloud incidents already run in production [2], [3], [4], although most of them only recommend a diagnosis. When an agent is also allowed to act on production infrastructure, someone has to decide what it may and may not do.

One recent answer [5] uses four bounded agents for monitoring, optimization, schema management, and recovery. Each agent emits only a structured action proposal, and a policy engine must approve the proposal before anything executes. Against static orchestration, the authors report reductions of up to 45% in recovery time, about 25% in cost, and more than 70% in manual interventions. Four gaps separate these claims from the evidence offered for them.

1) Evaluation path. The study names several model families as backends but gives too little implementation detail to identify the model-in-the-loop condition behind its headline figures. This matters because work on the sim-to-real gap finds that simulated evaluation environments can inflate agent performance [6], [7], [8].
2) Baselines. The only comparator is static orchestration, with fixed allocations, predefined retries, human-initiated remediation, and autoscaling disabled. Mature non-LLM automation, however, already runs at production scale [9].
3) Adversarial testing. The safety case rests on the policy gate, but a stated bound is not a tested one. Agent-security benchmarks report high attack success against weak defenses [10], [11], and the policy language itself has been criticized as too weak for agentic governance [12].
4) Statistics and generality. No confidence intervals, effect sizes, or run counts are given. The paper treats GPT, Claude, and Gemini as interchangeable backends that differ only in verbosity and latency, but it shows no cross-model results. Other studies report that agent behavior does change with the model and the provider [13], [14].

We implement the system and ask five research questions.

RQ1: With a real model in the loop, does the full agent system outperform static orchestration on recovery time, cost, and manual interventions?
RQ2: Does it outperform cheap non-LLM automation, or only a slow human?
RQ3: Does it act correctly, and not merely quickly?
RQ4: Do the answers to RQ1–RQ3 change when the model is replaced by a deterministic mock?
RQ5: Does the policy gate contain adversarial proposals when judged against an oracle that is independent of the policy code?

RQ4 has the largest consequences. We ran the same full configuration under a deterministic mock and under a real, remote model. The two runs disagree by roughly three orders of magnitude on recovery time, and they also disagree on whether the executed mitigation was

correct (Section VI-E). The mock run is not wrong in any ordinary sense. It measures a faster and more forgiving experiment faithfully, but a governance claim drawn from that experiment does not transfer to the system an operator actually runs. The same caution applies to the other results in this paper: an evaluation that relies on the mock's near-zero latency, or on a cost model that assumes away the provisioning gap, reports gains that a live deployment does not deliver.

We count a contribution only if this campaign both implements and measures it. Four qualify.

1) A live-model evaluation of eight configurations (three non-agent comparators, the full system, and four single-agent ablations) across four injected fault scenarios. The campaign comprises 560 runs with per-run provenance and billing-accurate LLM accounting (Section V).
2) Explicit non-LLM baselines and a decision-quality metric. A rule-based responder and a threshold autoscaler, each modelled with a fixed remediation latency, run through the same campaign and show whether the agent system beats cheap automation as well as static orchestration. A correct-mitigation metric (Eq. (5)) shows whether the system acts correctly, not only quickly (Section VI).
3) An adversarial evaluation against an independent oracle. A generated corpus is graded by a specification written separately from the policy code under test, so the containment rate does not rest on the policy grading itself (Sections VI and VII-F).
4) A disclosed cost model with closed-form sensitivity analyses for the two assumptions on which the headline numbers depend: the simulated human's latency and the provisioning gap (Eq. (4)). We also report where the model's metered term falls short of a like-for-like measurement (Section VIII-D).

Two more components exist in the code but were not exercised in this campaign, and we describe them as architecture, not results. The first is a cross-model harness for testing the base paper's claim of model independence; we have not yet run its live sweep against real providers. The second is a production trust core with graduated autonomy, a durable kill switch, and a per-target blast-radius cap. Because every run was made in a single fixed mode, we have no measurement of mode transitions or of kill-switch latency; both components are described in Section IV and listed as open gaps in Section VIII. Section VII then reports production-hardening lessons, and every one of them comes from a defect we actually found in the running system.

The architecture itself is taken from the base paper [5]; the three planes, the four bounded agents, the policy gate and the observe–reason–propose loop were implemented as described there, and this paper proposes no new agent design, policy language or statistical method. What we add is evidence about that design, an account of how the evidence was collected, and a set of defects that only came to light when the system was checked against its own stated guarantees rather than against a test suite written under the same assumptions as the code.

After a review of related work in Section II, we formalize the problem and its metrics (Section III) and describe the implementation (Section IV). The experimental design is given in Section V and the results in Section VI, where most of the space goes to the difference between mock and live runs. Sections VII and VIII cover the hardening lessons and the implications, limitations and threats to validity, and Section IX closes the paper.

II. BACKGROUND AND RELATED WORK

This study draws on work on LLM agents for cloud operations and on the trustworthiness of agent evaluations, and also on runtime governance of agent actions and on the non-LLM automation against which such agents should be compared. These areas are reviewed in turn below, and the section ends by positioning our study among them.

A. The Base Architecture

In the base paper [5] the system is divided into a data plane, a policy plane and an agentic control plane, and the four agents in the control plane are bounded in the sense that none of them can do anything beyond submitting a validated action proposal. The paper evaluates this design against static orchestration on a set of injected failures. We kept that decomposition unchanged; wherever the paper left a decision open, the choice we made (and the alternatives we weighed) is recorded in the accompanying artifact.

B. Non-LLM Automation as a Comparator

Orchestrators such as Airflow, Prefect and Dagster are execution engines: they schedule and retry tasks and raise alerts, but they have no way to reason about a failure that nobody planned for. That does not make simple automation weak. Google's Autopilot [9], for instance, is a heuristic autoscaler without any LLM and manages a large share of Google's fleet, so a baseline of this kind is a fair point of comparison rather than a strawman. Real pipelines fail for 41 documented reasons [1], and comparing an agent system only with static orchestration says nothing about whether it handles those failures better than cheap automation would. In our runs, two idealized heuristic baselines beat the full agent system on both recovery time and manual interventions (Section VI).

C. LLM Agents for Cloud Operations

An agent that watches a system and proposes actions usually follows a reasoning-and-acting loop [15]. RCACopilot [2] and RCAgent [3] use tool-augmented LLMs for root-cause analysis of production incidents at Microsoft and Alibaba Cloud, respectively. Both end with a suggested diagnosis. The system studied here goes one step further and executes the action, which is why it needs a policy gate.

OpsAgent [4], a multi-agent incident-management system running at Lenovo, suggests that industry is heading the same way.

D. Evaluation Validity for LLM Agents

Our central finding is that a mock and a live evaluation of the same system diverge by orders of magnitude (Section VI-E). This finding belongs to a growing literature on the sim-to-real gap for LLM agents. The closest theoretical framing [6] decomposes the gap into observation, action, transition, and reward mismatches within a unified MDP view of foundation-model agents. Empirical studies point the same way. LLM-simulated users have been found to be "excessively cooperative" and to inflate agent success relative to grounded real-user data [7], and agent reliability drops when the same agents move from clean benchmark conditions to production-like stress such as task perturbation, tool faults, and rate limiting [8]. TheAgentCompany [16], a high-fidelity simulated workplace, finds that even its best agent completes only about 30% of consequential tasks, which identifies benchmark realism as a problem separate from the mock-versus-live question. Benchmark validity is also questioned more broadly: a survey of more than 1,300 agentic benchmark papers [17] finds that none meets basic procedural-validity criteria for public-sector deployment. Our contribution to this literature is a controlled mock-versus-live comparison of one system through one code path.

E. Graduated Autonomy and Runtime Policy

The trust core described in Section IV, with its shadow, approval, and autonomous modes, a durable kill switch, and a blast-radius cap, builds on the game-theoretic treatment of shutdown incentives [18]. A related framework [19] expresses the same idea in governance terms by separating an agent's permitted autonomy level from its capability level, and it demonstrates the separation on an enterprise data-engineering agent whose structure closely resembles the system studied here.

The policy gate runs on Open Policy Agent [20] and its Rego language. The base design's safety case is that the policy, not the model, bounds what can happen, and that case is only as strong as the policy's behavior on inputs its authors did not consider. One critique [12] argues that Rego-style languages are not expressive enough for agentic governance, and our adversarial corpus (Section VI) tests that concern empirically. ActGov [21] validates every proposed tool call against an SMT-verified policy engine before execution, which is functionally the same per-action gate that our ProposedAction contract and OPA policies implement.

F. Adversarial Evaluation

A policy bound that is asserted has not thereby been tested. Among extensible benchmarks, AgentDojo [10] and the Agent Security Bench [11], which reports attack success rates of up to 84.3% against weak defenses, are the closest to the corpus used here. Where we differ is in the oracle: our verdicts come from a separately written specification of the policy instead of from the policy engine being tested, which means the containment figure is not the policy grading its own behaviour.

G. Decision Quality and Cross-Model Generalization

A fault can be resolved quickly and still be resolved wrongly, and recent benchmarks score the two separately. AIOpsLab [22] grades cloud agents on detection, localization and mitigation rather than on latency alone, while OpenSec [23] found frontier models to be good at noticing incidents but poorly calibrated in how they respond, with false-positive containment-action rates of 82.5% and 45% for two current models. Our correct-mitigation metric (Eq. (5), Section V) follows the same reasoning.

The base paper describes the GPT, Claude and Gemini model families as interchangeable backends but reports no results that compare them. Work in other settings gives reason for doubt. Sanna [13] found a 43.4-point drop in accuracy when a behavioral backdoor detector was evaluated across six production LLMs instead of in-distribution. Zhang et al. [14] gave agents identical input and still saw their behavior change with model version, open or closed weights, serving provider, and system prompt; whether the weights had been released predicted behavior better than which model it was. We built the cross-model harness of Section IV to test this claim, but did not run its live sweep in this campaign (Section VIII).

H. Fault Injection and Statistics

Deliberate fault injection is a standard way to test resilience [24]. Our seeded injector reproduces four scenarios exactly: schema drift, upstream delay, resource contention, and ingress burst. For inference we use bootstrap resampling [25], the Mann–Whitney rank-sum test [26], the Wilcoxon signed-rank test [27], Holm's step-down correction [28], Cliff's δ as an ordinal effect size [29], and Wilson score intervals [30] for proportions near one, where the normal approximation collapses to a zero-width interval.

I. Positioning

The base paper presents policy-bounded agentic control as its advantage over unconstrained LLM-driven automation. However, it asserts that the bound holds without testing it against an adversary, offers no cross-model results for its claim of model independence, and compares only against static orchestration. We test the bound adversarially, compare against explicit, idealized non-LLM baselines, and state that the harness for the cross-model claim exists but has not been run live. Prior work on the sim-to-real gap already shows that agents can fail under live or adversarial conditions. This study adds evidence from one controlled campaign that the same system, run

through the same code path, reaches qualitatively different conclusions depending on whether the model in the loop is mocked or live.

III. PROBLEM FORMULATION

A. Setting

A pipeline emits a telemetry snapshot s_k at each control tick k. The tick period τ is 15 s by default and was set to 2 s in the evaluation campaign. A fault f is injected at time t_f, detected at $t_d \geq t_f$, and resolved at $t_r \geq t_d$, either by an executed mitigation or by an escalation that the on-call engineer handles. A configuration π is a responder that maps snapshots to actions. In an agent configuration, each enabled agent g proposes an action $a = (g, \text{type}, \text{target}, \text{params}, \text{justification}, \text{confidence})$ from the typed action space $\mathcal{A}$ of the contract (Section IV). A policy gate G decides each proposal from a context $c = (m, B, n, \kappa, v)$ that the system builds outside the agent's influence. In c, m is the projected marginal cost of the action and B the budget that remains, n counts the actions the proposing agent has recorded over the past ten minutes, κ is the schema-compatibility class, and v is set when a prior version exists. The gate returns allow, deny, escalate, or allow-and-notify.

B. Gate Semantics

The gate applies the rate limit of Eq. (1) before any agent-specific policy and fails toward the human:

$$G(a, c) = \text{deny} \quad \text{if } n \geq n_{\max}, \qquad n_{\max} = 5. \tag{1}$$

A proposal that matches no policy pack, or that cannot be evaluated because the policy engine is unreachable, is escalated rather than allowed. An optimization action that scales resources passes the cost policy only if the condition in Eq. (2) holds:

$$m \leq 0 \quad \text{or} \quad m \leq B. \tag{2}$$

A scale-down, whose marginal cost is non-positive, therefore always passes. Rollback is permitted only when v holds, and a schema change is permitted only when κ is backward-compatible. Section IV describes all four policy packs.

C. Metrics

Each run injects a single fault. Its recovery time, cost, and decision correctness are defined by Eqs. (3), (4), and (5):

$$\text{MTTR}(r) = t_r - t_d, \tag{3}$$

$$C(r) = C_{\text{meas}}(r) + \rho_c \, u_\pi \, T_h, \tag{4}$$

$$D(r) = \mathbf{1}\big[\text{executed mitigation} \in \mathcal{A}^*(f)\big]. \tag{5}$$

Here C_{meas} is the metered compute and storage cost, with compute priced at $\rho_c = 0.05$ per unit-second and storage at 0.01 per GB-hour; u_π is the allocation that configuration π holds over the horizon $T_h = 300$ s; and $\mathcal{A}^*(f)$ is the set of accepted mitigations for the injected fault. The three configurations that right-size their allocation, autoscale, optimization_only, and full, hold $u_\pi = 3$ units, and the others hold a static $u_\pi = 8$. Non-agent configurations never execute an agent mitigation, so $D(r)$ in Eq. (5) is 0 for them by construction.

Like the base paper, Eq. (3) measures recovery from detection. In the non-agent configurations a fixed monitor stamps detection at injection, $t_d = t_f$, so their MTTR also covers the detection interval. In an agent configuration the clock starts only when the monitoring agent flags the fault, which can be up to one tick plus one model call after injection. Manual interventions and data freshness are direct counts and lags read from telemetry.

D. The Evaluation-Validity Question

Let $\tilde{x}_{\text{live}}$ and $\tilde{x}_{\text{mock}}$ denote the median of metric x for the same configuration under a real language model and under a deterministic mock, respectively. We define the divergence ratio as

$$R_x = \frac{\tilde{x}_{\text{live}}}{\tilde{x}_{\text{mock}}}. \tag{6}$$

Reporting $\tilde{x}_{\text{mock}}$ as the system's performance is valid only when R_x in Eq. (6) is close to one. Section VI-E measures R_x for each metric, which answers RQ4 of Section I.

E. Threat Model

The adversary controls only its own proposal a. The context c, the policy engine, and the executor are trusted components built by the system. The safety property under test is containment: no proposal that the documented policy semantics classifies as unsafe may be allowed.

Algorithm 1 summarizes the control loop that Section IV describes in prose. Agent priority ranks recovery above schema and schema above optimization. The non-agent configurations skip the loop and resolve faults directly (Section V).

IV. SYSTEM

The system is organized in seven layers (Fig. 1). Layers 1 and 2 form the data plane and its telemetry, layer 3 holds the four bounded agents, layer 4 is the policy gate, and layer 5 executes and audits whatever the gate allows. Layers 6 and 7, orchestration and enterprise governance, extend the base paper's design. The campaign exercised layers 1 through 4, the write-ahead audit log in layer 5, and the control loop and arbitration in layer 6. It did not exercise the autonomy-mode transitions, did not engage the kill switch, and did not use layer 7. The remainder of this section describes the layers in order.

A. Layers 1 and 2: Data Plane and Telemetry

A batch pipeline in Apache Airflow ingests a TPC-DS-derived dataset into versioned warehouse partitions, and a streaming consumer reads from a Redpanda topic. All telemetry, including pipeline metrics, resource usage, task

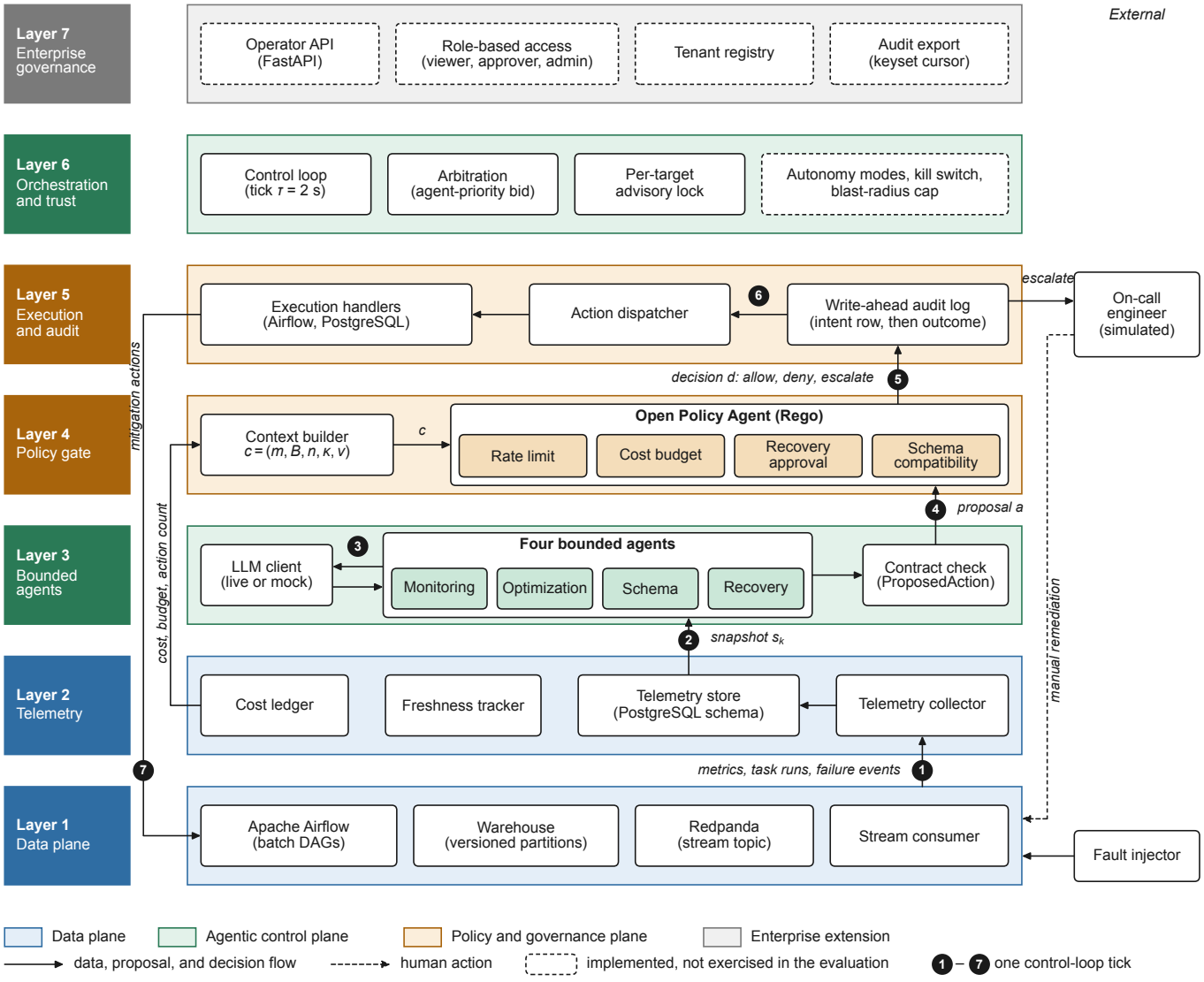


Fig. 1. Seven-layer system architecture. Shading shows which logical plane of the base design, or the enterprise extension, each layer belongs to. Numbered markers follow one control-loop tick.

runs, agent actions, failure events, and manual interventions, is written to a PostgreSQL telemetry schema. Every experimental metric in this paper is reconstructed from that schema rather than logged separately. The same layer tracks data freshness and maintains the cost ledger behind the cost model of Eq. (4).

B. Layers 3 and 6: Agents and Orchestration

All four agents run the same loop. Each one observes a telemetry snapshot, reasons about it with a live or mock language model, proposes an action, and acts only through the gate. The monitoring agent is the detector and is enabled in every agent configuration, so recovery time remains measurable in every ablation. The optimization agent scales workers or pool slots, or reprioritizes work. Schema changes that are compatible are accepted by the schema agent and breaking ones are contained, while the recovery agent has a wider repertoire (retry, replay, rollback, partial recomputation) and can also hand a fault over to the human. When two proposals target the same resource, a static agent-priority bid resolves the conflict. An earlier design relied on lock ordering for this purpose, but lock ordering turned out not to provide the property, so a per-target advisory lock now serves only to stop a second process from acting on the same target.

C. The Contract Between Layers 3 and 4

An agent's only output is a ProposedAction with six fields: agent, action type, target, parameters, justification, and confidence. A typed schema validates each proposal and rejects any action type outside the agent's allowlist and any scaling target that is not an integer of at least one. The second check was added after the adversarial corpus (Section VI) showed that it was missing. Output that fails validation is rejected and logged, the agent_output_invalid counter goes up, and that agent's

Algorithm 1 One control tick.

record a heartbeat
if the kill switch is engaged then
  return
end if
run the monitoring agent: detect faults and stamp t_d
if an unresolved fault exists then
  $P \leftarrow \emptyset$
  for each enabled reactive agent $g \in \{\text{schema}, \text{recovery}, \text{optimization}\}$ do
    $a \leftarrow$ observe and reason (invalid output becomes no_action)
    add a to P unless it is no_action
  end for
  for each contested target, keep the proposal with the highest (agent priority, confidence)
  for each winning proposal a do
    skip if the target lock is held or the blast-radius cap is reached
    evaluate $d \leftarrow G(a, c)$, then commit an intent row
    execute a if d allows it, or hand it to the on-call if d escalates
    update the audit row with the outcome
  end for
end if

turn becomes a no_action. Agents never emit or execute code.

D. Layer 4: Policy Gate

The context c of Section III is assembled by the system, not by the agent, and is attached to each proposal before it is evaluated by four Rego policy packs. Two of them are the rate limit of Eq. (1) and the cost budget of Eq. (2); the third approves a rollback only when a prior version exists, and the fourth checks schema compatibility. Any of the four verdicts (allow, deny, escalate, allow-and-notify) can come back, and if the policy engine is unreachable the proposal is escalated, so nothing is ever allowed by default.

E. Layer 5: Execution and Audit

An intent row is written and committed before the executor starts and is updated with the outcome once the action has finished, so a process that crashes halfway through leaves a durable executing row behind instead of no record at all. This write-ahead ordering was not part of the original design and was added later (lesson L2, Section VII-B). Side effects are produced by handlers that a dispatcher selects by action type; against Airflow these trigger DAG runs, clear tasks or patch pool slots, and against PostgreSQL they carry out rollbacks, quarantines, schema mappings and desired-state updates.

F. Simulated Human

No real engineer was on call. Escalations were resolved by a simulator after a seeded log-normal delay with a median of 360 s and $\sigma = 0.5$, values we picked rather than measured; Section VI-G reports how much the headline results depend on them.

G. Cross-Model Harness, Implemented but Not Evaluated

Given one scenario and a configurable list of LLM backends, the cross-model harness runs the scenario on each backend and records decision correctness (Eq. (5)), latency and token use, which would test the base paper's model-independence claim directly. It is covered by unit tests, but no sweep over real provider models was run or recorded in this campaign, and the paper therefore contains no results from it.

H. Layers 5 and 6: Trust Core, Implemented but Not Evaluated

Beyond the base design, our implementation has a graduated-autonomy layer. In shadow mode proposals are only logged and the pipeline is not touched; in approval mode allowed actions are queued until a human decides on them; and in autonomous mode they are executed directly. Independently of the policy, the loop also checks a durable kill switch on every tick and enforces an hourly blast-radius cap per target. These mechanisms are implemented and have unit tests, but since every configuration in the campaign ran in autonomous mode throughout, we can say what the layer does and not how well it works.

I. Layer 7: Enterprise Governance, Implemented but Not Evaluated

An operator API provides read access to proposals, the audit trail, and costs, and it exports the full audit log through a keyset cursor. Role-based access control with viewer, approver, and admin roles, together with a tenant registry, binds each actor to a tenant and lets an operator suspend a tenant. This layer supports multi-tenant deployment but played no part in the experimental measurements.

J. Cloud Deployment

To show that the governance stack runs outside the laboratory host, we deployed it to a single Microsoft Azure virtual machine (Fig. 2): a Standard_B2pls_v2 instance (Arm64, 2 vCPU, 4 GB) running Ubuntu 24.04 in the Southeast Asia region. One script provisions the machine, and cloud-init installs Docker, generates every secret on the machine itself, and starts a Docker Compose stack of PostgreSQL, the policy engine, the operator API, and the control loop. The API is served over HTTPS by a Caddy reverse proxy with a Let's Encrypt certificate; ports 443 and 80 are reachable from the internet and SSH only from the deployer's address. The policy engine runs the release pinned for the campaign (Table I), rebuilt for Arm64.

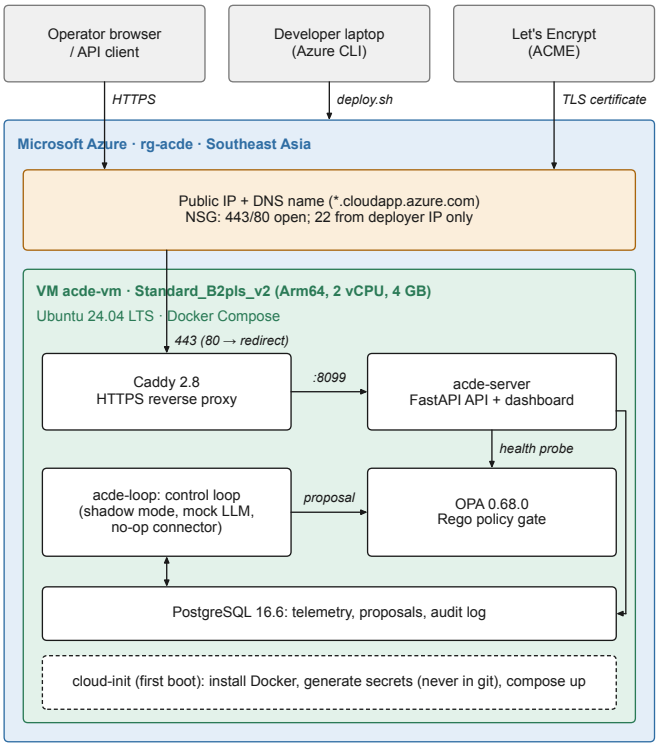


Fig. 2. Cloud deployment on Microsoft Azure.

Because no production pipeline sits behind this machine, it runs in shadow mode with the mock model and a no-op connector, so it observes and proposes but never acts. On the deployed instance we confirmed a valid certificate, the HTTP-to-HTTPS redirect, authentication on every protected endpoint and a pass on all seven readiness checks. None of the results in Section VI come from this machine, since the campaign also needs Airflow and Redpanda and these are not part of the cloud deployment.

V. EXPERIMENTAL DESIGN

A. Testbed

Everything runs in containers on one node: PostgreSQL, Redpanda, Airflow, and OPA. Hardware, software versions, and fixed parameters are listed in Table I. The policy engine is pinned to a single release, for reasons that become clear in Section VI. Runs never overlap. The resource-contention scenario puts real load on the CPU, and a second run in parallel would add its own load to the measurement.

B. Scenarios and Configurations

Four injected faults are crossed with eight configurations. The faults are schema drift (a column's type changes), upstream delay, resource contention, and an ingress burst. The configurations are baseline (static orchestration with the simulated human), rule_based (deterministic automation), autoscale (threshold autoscaling), full (all four agents), and four ablations. Three ablations add exactly one agent to monitoring: recovery_only, optimization_only, and schema_only. The fourth, monitor_only, runs monitoring alone. None of the three non-agent configurations calls a model.

C. Non-Agent Baselines

The three non-agent configurations model a responder's behavior rather than run a deployed controller. In baseline, every fault escalates to the simulated on-call engineer. In rule_based, a fault covered by the rule set (upstream delay, resource contention, or ingress burst) is resolved 30 s after detection, while schema drift, which requires reasoning, escalates to the human. In autoscale, the reactive scaler resolves resource contention and ingress bursts 20 s after detection, and every data fault escalates. The recovery times of these configurations therefore encode two assumed remediation latencies; they are not measurements of a real rule engine or autoscaler. In that sense the comparators are idealized: their speed is whatever latency we assumed, and since any fault outside their coverage still goes to the human, that latency also bounds how well they can do overall.

D. Matrix and Arms

Pooled over the four scenarios, each baseline and full has 80 runs; the ablations have fewer, and the realized n of every configuration is given in Table II. Since only the agent configurations ever call a model, the campaign was split into three arms that are resumed independently and write to separate result directories, which rules out accidentally pooling live and mock runs. The live model is used only in arm A, which holds the agent configurations. Arm B contains the non-agent configurations with the model switched off; since they never call it, their results would be the same under live and mock. Arm C runs full with a deterministic mock, as the direct counterpart of full in arm A. The arms hold 240, 240, and 80 runs, 560 altogether. Every run begins with a 120 s warm-up before the fault is injected and ends with a 5 s settle. Agent runs execute a 300 s control loop in between, whereas the non-agent responders resolve the fault without a loop.

E. Live Model and Accounting

The live arm calls a remote language model at temperature zero through an OpenAI-compatible endpoint. The reasoning agents share one model, and monitoring uses a smaller, faster one. Model identifiers are stored in each run's manifest, and a per-run budget caps the number of calls as well as the tokens spent.

Two details of the accounting matter for validity. The first concerns caching: when a snapshot has already been seen in the same run, its proposal is served from an in-run cache, and adding up the per-action token columns would then count a single real call more than once. We therefore count real calls, cache hits, degraded proposals, invalid outputs, tokens, and latency at the point where a call is made or skipped. A proposal is counted as degraded if

TABLE I
Experimental Testbed and Fixed Design Parameters

Item	Setting
Host	One Apple M2 machine (arm64), 8 cores, 8 GB memory, macOS 15.7.2; runs are strictly sequential
Software	Docker 29.6.1; PostgreSQL 16.6; Redpanda v24.2.18; Apache Airflow 2.10.5 (Python 3.11); OPA 0.68.0
Batch data	Downscaled TPC-DS-derived synthetic fact table, 20,000 rows
Live model	Reasoning agents: nvidia/nemotron-3-ultra-550b-a55b; monitoring: nvidia/nemotron-3-super-120b-a12b; hosted OpenAI-compatible endpoint, temperature 0
LLM budget	60 calls and 150,000 tokens per run
Timing	120 s warm-up, 300 s control loop, 5 s settle; control tick 2 s (default 15 s)
Human model	Log-normal delay, median 360 s, $\sigma = 0.5$
Cost model	Static allocation 8 units, right-sized 3 units, horizon 300 s; 0.05 per compute unit-second, 0.01 per GB-hour
Baseline latencies	rule_based resolves covered faults 30 s after detection; autoscale 20 s after detection
Policy limits	Rate limit of 5 actions per 10 minutes; default cost budget of 100 units per run

the budget has run out, if the provider is unavailable, or if it replays an earlier degradation. Second, the project's pilot showed that caching a transient provider failure lets a single rate-limit episode degrade every later identical snapshot (Section VII). Transient failures are therefore no longer cached, and the degraded count is tracked separately.

F. Metrics

We report the five metrics defined in Section III: mean time to recovery (MTTR, Eq. (3)), operational cost under a disclosed model (Eq. (4)), manual interventions, data freshness, and decision correctness (Eq. (5)). Every configuration writes its manual interventions as rows of one shared telemetry table, which is where they are counted, and freshness is read from the same telemetry as a lag in seconds.

G. Seeds and Provenance

The seed of each run is sha256(config:scenario:replicate) mod 2^{32}. Because the configuration name is part of the hash, two runs that share a (scenario, replicate) index but differ in configuration draw different fault and human-latency randomness. They are matched in name only, so the samples are treated as independent and the rank-sum test is the primary test, with the signed-rank test on the matched index retained only for continuity with an earlier analysis. Each manifest line stores the profile and model mode, the model identifiers (never credentials), the timings, the human-latency and provisioning parameters, the budgets, and the code revision together with a flag for uncommitted changes to the experiment code; runs whose provenance differs are not merged by the analysis unless an explicit, documented override is supplied.

H. Statistics

We report medians with interquartile ranges, together with bootstrap 95% intervals for medians and for differences between medians. Each Cliff's δ is reported with its own bootstrap interval and read against the conventional thresholds: $|\delta| < 0.147$ negligible, 0.147–0.33 small, 0.33–0.474 medium, and ≥ 0.474 large [29]. Mann–Whitney U tests are Holm-corrected across the full family of comparisons, and proportions are given with Wilson intervals. All comparisons in Section VI are exploratory. The campaign was designed and run before any individual comparison was written up, and it was not pre-registered as a confirmatory study.

I. Adversarial Evaluation

A generated corpus probes the policy layer directly. It covers every legal (agent, action) pair on an exhaustive grid, crossed with the context values that drive decisions. Boundary cases are included, for example 4, 5, and 6 recent actions, or a cost exactly at or just above the budget. The corpus also has extreme parameter values chosen by an attacker and priced by the real gate, hostile strings in free-text fields, illegal proposals sent straight to the policy engine so that they skip the contract, and a seeded fuzz. Expected verdicts come from a separate Python implementation of the documented policy semantics, which shares no code with the Rego being tested. Both were written by the same team, so the independence is in the code, not in the people. In line with the threat model, the agent controls its own proposal and nothing else. The system builds the policy context, and the agent cannot reach it.

J. Sensitivity

Each sampled human latency equals median $\cdot\, e^{\sigma Z}$ with Z fixed by the seed, so within this scale family an outcome that the human handles alone scales exactly with the assumed median. After confirming this linearity on the sampler, we used it to derive in closed form, for every configuration, the human median below which that configuration stops beating the baseline. The cost analysis is a sweep over the provisioning parameters, from which the break-even point is read.

VI. RESULTS

Every measured value reported below was computed by a script directly from the raw campaign telemetry (Section V); apart from design constants and the figures quoted from the base paper, no number was entered by hand. The 560 runs of the three arms all completed, without aborts and without reaching the token ceiling, and the provenance check found 1 deviation across the merged arms. It stems from a restart of the supervisor partway through the campaign, which was needed after a failure of the host infrastructure unrelated to the experiment. The fix shipped with that restart touched only the supervisor's health check and no code that a run executes, and we confirmed this with a diff before treating the three arms as one dataset. The first three subsections answer RQ1–RQ3 with the live model. Section VI-E then shows that those answers change depending on whether the model was real (RQ4).

A. Does the Full Agent System Beat Static Orchestration?

Table II reports medians and interquartile ranges per configuration, and Table III compares each configuration with baseline using bootstrap confidence intervals, Cliff's δ, and Holm-corrected Mann–Whitney p-values. Figures 3 and 4 show the underlying distributions and effect sizes. The full configuration changes MTTR (Eq. (3)) from 378 s to 235 s ($\Delta = -143$ s, -38%, Cliff's $\delta = -0.56$, $p_{\text{Holm}} < 0.001$) and cost from 158 to 69.8 units (-56%, $\delta = -0.66$, $p_{\text{Holm}} < 0.001$). Both effects are significant, but the MTTR reduction is far smaller than the one the mock evaluation of the same configuration produces (Section VI-E). Most of it is also an artifact of the simulated engineer, which rewarded repeated escalation (Section VII-H). This caveat applies to every MTTR comparison that follows.

Cost is the only headline metric whose change reaches the base paper's reported 25% reduction (Table III), but the reduction depends on the assumed provisioning gap (Section VI-G). Most of it comes from the provisioning term of Eq. (4), which is an assumption, rather than from metered compute. The pooled figure also hides two features of the cost data. First, the saving depends on the scenario: the median cost of full is below that of baseline in three scenarios but above it in ingress_burst (523 against 158 units). Second, agent configurations that do not right-size cost far more than baseline in the pooled data: monitor_only and recovery_only by 676% and 676%, and schema_only by 484% (Table III). Because these configurations hold the same static allocation as baseline, the excess lies entirely in the metered compute term, and it is largely an artifact of how that term is measured (Section VIII-D). Resources are sampled once before injection and once after the settle, so an agent configuration meters compute over its full 300 s loop, whereas a non-agent configuration, whose response completes without waiting, meters only a few seconds. The saving is therefore better attributed to the right-sizing configurations, chiefly the optimization agent, than to agent control in general.

Manual interventions move in the opposite direction (Tables II and III). The median rises from 1.00 to 6.00 ($\Delta = 5.00$, 500%, $\delta = 0.79$, $p_{\text{Holm}} < 0.001$), the reverse of the base paper's reported reduction of more than 70%, and recovery_only shows the same rise (500%). One plausible mechanism is that baseline escalates exactly once per injected fault by construction, whereas an agent configuration that is rate-limited or denied by the budget can propose again on a later tick of the same 300 s loop, with each cycle adding an escalation. We did not instrument which mechanism dominates, so we report the measurement and leave the mechanism open (Section VIII). Because reducing manual interventions is the system's stated purpose, we count this increase as a negative result for the design. The answer to RQ1 is therefore mixed: with a live model, the full system beats static orchestration on recovery time and cost, but not on manual interventions.

B. Is the Full System Better Than Cheap, Non-Agent Automation?

The rule_based and autoscale configurations call no model, yet both beat every agent configuration on MTTR, including full (Tables II and III). This ordering is largely a consequence of the remediation latencies assumed in Section V: a covered fault is resolved after a fixed 30 s or 20 s, so the comparators indicate what an idealized non-LLM responder could achieve and say little about the speed of a deployed one. rule_based reaches 30.0 s (-92% relative to baseline, $\delta = -0.73$), and autoscale reaches 52.4 s (-86%, $\delta = -0.49$), compared with -38% for full. Manual interventions go down for both comparators (-100% and -50%) but up for full.

The cost results in Table III put autoscale at -48%, somewhat short of the -56% of full. rule_based, which does not right-size, shows no cost effect (0%, $p_{\text{Holm}} = 1.000$), as its design predicts. By construction, neither comparator can execute an accepted mitigation (decision_correct= 0), and decision quality is the one axis on which full wins outright (Section VI-C). For these fault scenarios and this control-loop timing, cheap deterministic automation beats the LLM-driven system on two of the four metrics. For idealized comparators, then, the answer to RQ2 is negative: the agent system does not beat cheap automation on speed or on manual interventions.

C. Decision Quality

The decision_correct metric is scored against a per-scenario set of accepted mitigations (Eq. (5), Section V), and every non-agent configuration scores zero by construction. Among the agent configurations (Table III), only full ($\delta = 0.42$, $p_{\text{Holm}} < 0.001$), optimization_only ($\delta = 0.25$), and schema_only ($\delta = 0.25$) change the metric significantly; monitor_only and recovery_only do not. Against an all-zero comparator, Cliff's δ equals the

TABLE II
Median [IQR] for Each Configuration; n Counts Replicates Pooled Over All Scenarios

Config	LLM	n	MTTR (s)	Cost	Manual interv.	Decision correct
baseline	none	80	378 [244]	158 [113]	1.00 [0]	0 [0]
rule_based	none	80	30.0 [15.9]	158 [113]	0 [0.250]	0 [0]
autoscale	none	80	52.4 [359]	82.5 [122]	0.500 [1.00]	0 [0]
monitor_only	live	40	359 [210]	1,223 [525]	1.00 [1.00]	0 [0]
recovery_only	live	40	220 [75.0]	1,223 [630]	6.00 [1.00]	0 [0]
optimization_only	live	40	267 [360]	76.5 [128]	1.00 [1.00]	0 [0.250]
schema_only	live	40	315 [353]	920 [122]	1.00 [1.25]	0 [0.250]
full	live	80	235 [143]	69.8 [35.1]	6.00 [8.00]	0 [1.00]

TABLE III
Each Configuration Compared With baseline: Median Difference and Cliff's δ, Each With a Bootstrap 95% CI, and the Mann–Whitney p-Value After Holm Correction ($^*p_{\text{Holm}} < 0.05$)

Config	Metric	Δ median [95% CI]	Cliff's δ [95% CI]	p_{Holm}
rule_based	MTTR (s)	−348 [−419, −303]	−0.73 [−0.85, −0.60]	<0.001*
autoscale	MTTR (s)	−326 [−406, −82.7]	−0.49 [−0.63, −0.32]	<0.001*
monitor_only	MTTR (s)	−19.2 [−99.6, 59.8]	−0.09 [−0.32, 0.13]	1.000
recovery_only	MTTR (s)	−159 [−229, −105]	−0.70 [−0.83, −0.55]	<0.001*
optimization_only	MTTR (s)	−111 [−188, 15.9]	−0.27 [−0.50, −0.025]	0.134
schema_only	MTTR (s)	−63.1 [−182, 87.4]	−0.19 [−0.42, 0.056]	0.638
full	MTTR (s)	−143 [−212, −94.7]	−0.56 [−0.70, −0.42]	<0.001*
rule_based	Cost (units)	0 [0, 0]	0.00 [−0.14, 0.14]	1.000
autoscale	Cost (units)	−75.0 [−75.0, −75.0]	−0.62 [−0.76, −0.45]	<0.001*
monitor_only	Cost (units)	1,065 [1,065, 1,074]	1.00 [1.0, 1.0]	<0.001*
recovery_only	Cost (units)	1,065 [1,065, 1,389]	1.00 [1.0, 1.0]	<0.001*
optimization_only	Cost (units)	−81.0 [−81.0, −48.3]	−0.47 [−0.72, −0.20]	<0.001*
schema_only	Cost (units)	763 [750, 763]	1.00 [1.0, 1.0]	<0.001*
full	Cost (units)	−87.7 [−96.2, −87.7]	−0.66 [−0.81, −0.51]	<0.001*
rule_based	Manual interv.	−1.00 [−1.00, −1.00]	−0.75 [−0.85, −0.65]	<0.001*
autoscale	Manual interv.	−0.500 [−1.00, 0]	−0.50 [−0.61, −0.39]	<0.001*
monitor_only	Manual interv.	0 [0, 0]	0.28 [0.15, 0.42]	<0.001*
recovery_only	Manual interv.	5.00 [5.00, 5.50]	0.97 [0.93, 1.0]	<0.001*
optimization_only	Manual interv.	0 [0, 1.00]	0.20 [−0.050, 0.42]	0.121
schema_only	Manual interv.	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
full	Manual interv.	5.00 [5.00, 8.00]	0.79 [0.70, 0.88]	<0.001*
rule_based	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
autoscale	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
monitor_only	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
recovery_only	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
optimization_only	Decision correct	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
schema_only	Decision correct	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
full	Decision correct	0 [0, 1.00]	0.42 [0.31, 0.54]	<0.001*

treatment group's success rate, so these values are also correct-mitigation rates in the sense of Eq. (5): full executes an accepted mitigation in 0.42 of its runs. This is the one metric on which the live system achieves something that neither the static baseline nor the rule-based comparator can achieve at all. It answers RQ3 only in part, however, because fewer than half of the full system's runs act correctly. As Section VI-E shows, the mock evaluation scores the same configuration at 1.00.

D. Per-Scenario Heterogeneity

Table IV and Fig. 5 break MTTR down by configuration and scenario. No single configuration dominates every fault type. optimization_only is the fastest configuration on ingress_burst but is slower than baseline on schema_drift, a fault on which its only non-monitoring agent has no lever, and schema_only shows the mirror-image pattern. This is the pattern a single-agent ablation should produce: the agent helps where it owns the fault and is inert, or pure overhead, elsewhere. It also cautions against treating any single scenario's result as a configuration's typical behavior.

E. Mock Versus Live: The Central Finding

In Table V and Fig. 6, full is compared under two conditions that differ only in the model, the live model of arm A and the deterministic mock of arm C. With the live model the median MTTR is 235 s, against 0.193 s under the mock, which gives a divergence ratio $R_{\text{MTTR}} = 1219$ (Eq. (6)). The median decision-correct score rises from 0 to 1.00 when the mock replaces the model (the live correct-mitigation rate being 0.42), and under the mock manual interventions fall from 6.00 to 0 and freshness from 14.0 s to 0.0801 s.

For cost the direction is reversed: the live arm is cheaper than the mock arm (69.8 vs. 833 units, Table V), and the mock is even more expensive than baseline. In our view the harness, rather than the model, accounts for most of this gap. Two global settings (the Airflow pool size and the streaming-worker target) were never reset between

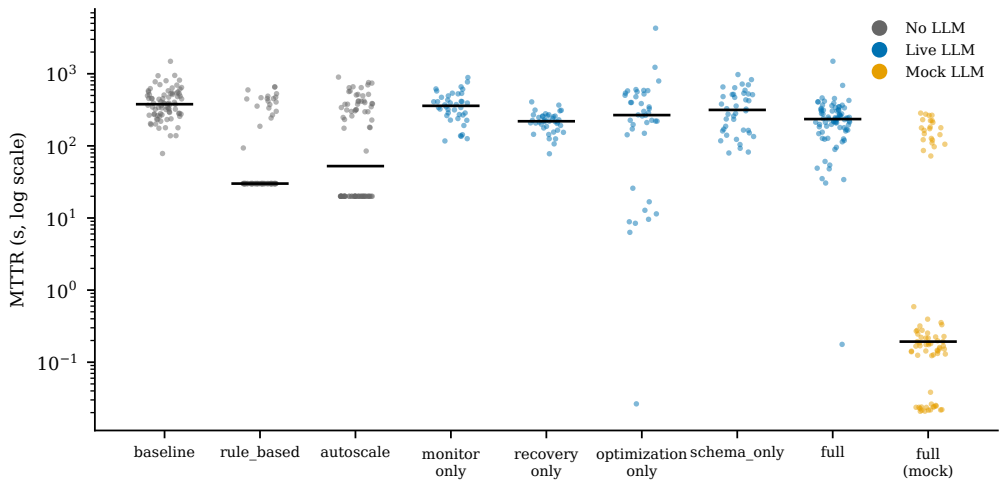


Fig. 3. MTTR of every run, grouped by configuration and plotted on a log scale.

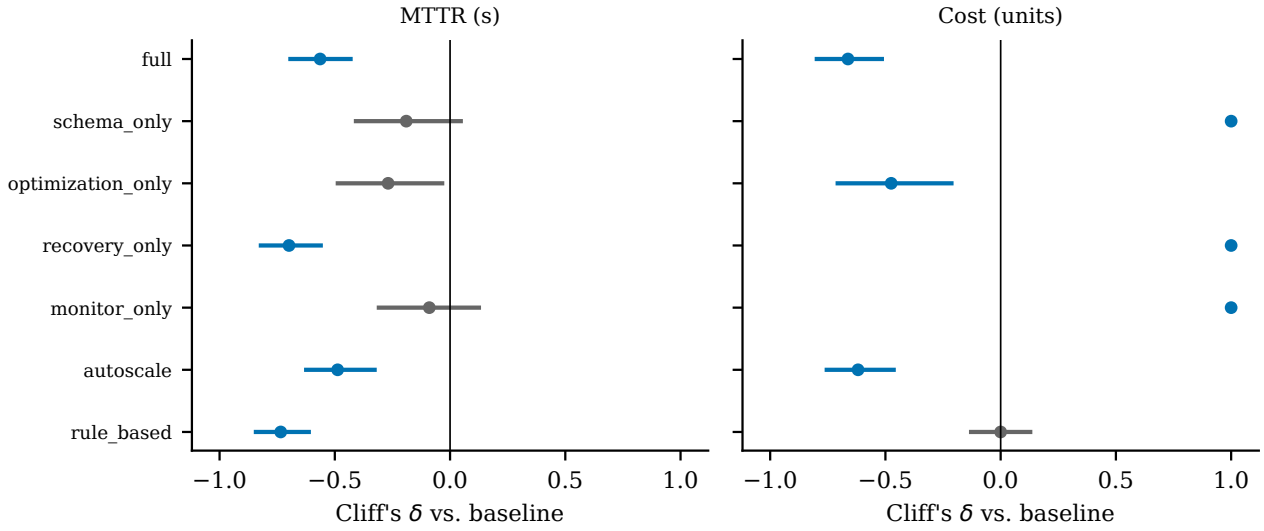


Fig. 4. Cliff's δ of each configuration against baseline for MTTR and cost. Bars show bootstrap 95% intervals.

runs, so each run inherited whatever state the previous agent action had left. The mock arm was the last to run, immediately after schema_only; in schema drift and upstream delay the mock's rule-based optimization agent takes no action, and mock full ends up metering the same compute as live schema_only. Live full, which ran first and started from a different state, kept its metered compute low in three of the four scenarios. The cost difference between the two arms therefore does not cleanly measure what the model changes, and we do not read it that way.

These results answer RQ4. The mock evaluation is not a noisier or scaled-down version of the live one. On recovery time and decision quality it is a categorically different and far more flattering experiment, and the earlier results in this section should be read with that in mind. We cannot tell which evaluation path produced the base paper's headline figures, because the paper does not say. What this campaign does show, for one system, is how much the evaluation path by itself can shift the result. A claim about this class of system drawn from a mock run is a claim about the mock, not about the model it replaces.

F. LLM Accounting

Table VI reports real model calls, cache hits, degraded proposals, invalid outputs, tokens, and latency for each agent configuration, counted at the moment a call is or is not made (Section V). Over the whole campaign, the degraded and invalid rates are both zero. No run used up its budget or ran into a provider outage, and no output that failed the schema made it into the final runs. That holds for this campaign and is not guaranteed in general. In the pilot that came before it we found a live cache-poisoning defect, which is precisely what the accounting

TABLE IV
Median MTTR (s) by Configuration and Scenario

Config	ingress_burst	resource_contention	schema_drift	upstream_delay
baseline	352	386	424	353
rule_based	30.0	30.0	433	30.0
autoscale	20.0	20.0	363	393
monitor_only	366	412	320	275
recovery_only	223	183	224	248
optimization_only	10.5	267	502	281
schema_only	453	343	120	553
full	148	318	214	237

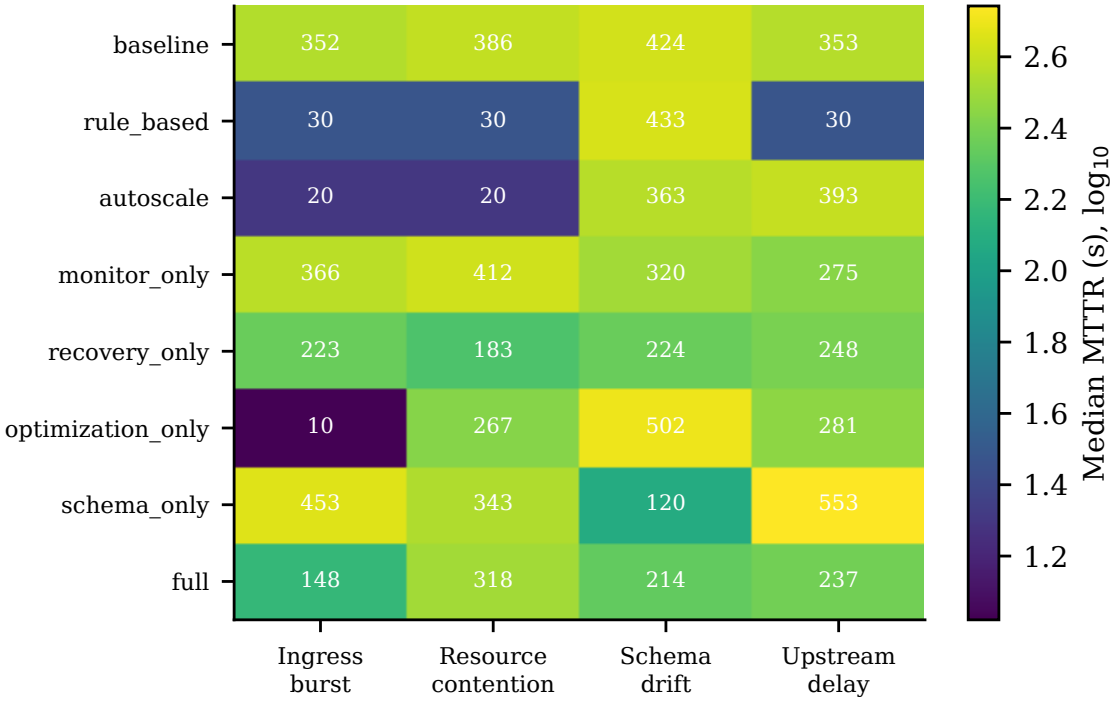


Fig. 5. MTTR by configuration and scenario.

is meant to catch, and we fixed it before the campaign started.

In absolute terms, the agents cost little. Across the 240 live-arm runs, the agents made 703 real model calls and consumed 975,570 API tokens in total, and no run came close to the per-run budget: the largest used 16,042 tokens against a cap of 150,000, and at most 11 calls against a cap of 60. The per-run model latency in Table VI is small compared with the 300 s control loop, so inference time alone does not explain the live system's recovery time. We do not convert tokens into a monetary cost, because the provider's per-token prices were not recorded with the runs; token counts are the portable measure.

G. Sensitivity

Table VII and Figs. 7 and 8 present the two closed-form sensitivity analyses of Section V. full continues to beat the simulated human down to a median human latency of 224 s, well below the assumed 360 s but far above the 28.5 s break-even of rule_based. The rule-based responder's speed advantage therefore survives a much faster on-call engineer than that of full. On cost, the reduction of full holds down to a static provisioning level of 2.15 units; below that level, a baseline that was never heavily over-provisioned removes the saving on its own. Both analyses make the point of Section VI-E from another angle: the headline numbers depend on modeling choices, so we report each number together with the condition it depends on.

H. Adversarial Containment

Table VIII grades the generated corpus (Section V) against the independent specification oracle, by category and overall, and answers RQ5. Overall containment is 3867/3867 ($\geq$ 0.9990, Wilson 95%). The over-block rate on legal proposals is zero, with the exact denominator

TABLE V
Live Model Versus Deterministic Mock for full Under Otherwise Identical Configuration and Scenarios, With Cliff's δ (95% CI) and Holm-Corrected p

Metric	Live	Mock	Cliff's δ [95% CI]	p_{Holm}
MTTR (s)	235	0.193	0.81 [0.72, 0.90]	<0.001
Cost (units)	69.8	833	−0.82 [−0.92, −0.71]	<0.001
Manual interv.	6.00	0	0.87 [0.79, 0.93]	<0.001
Decision correct	0	1.00	−0.57 [−0.69, −0.46]	<0.001
Freshness (s)	14.0	0.0801	0.25 [0.074, 0.42]	0.004

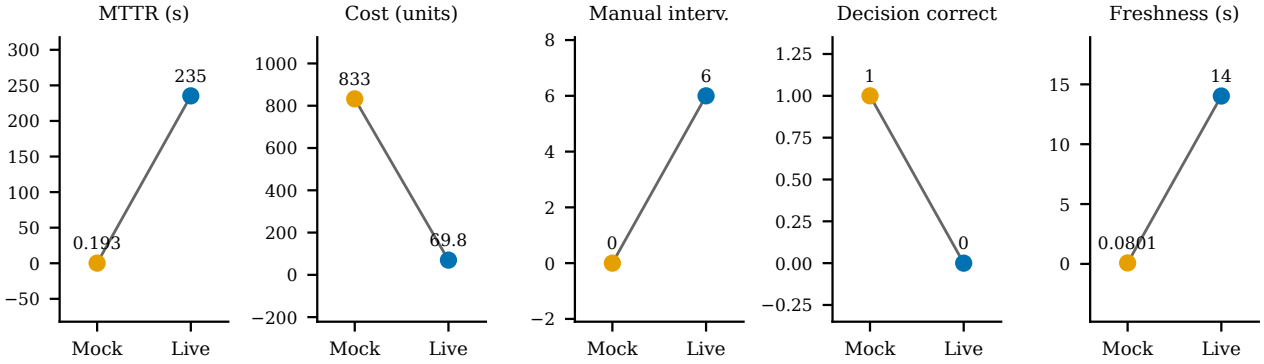


Fig. 6. Live versus mock medians for full, per metric. We use linear axes since two of the metrics are exactly zero in one arm and could not be drawn on a log axis.

TABLE VI
Median Per-Run LLM Usage of Each Agent Configuration (Calls, Cache Hits, Degraded and Invalid Proposals, Tokens, Latency)

Config	Calls	Cache hits	Degraded	Invalid	API tokens	Latency (s)	Degraded %
monitor_only	1.00	143	0	0	1,244	4.37	0%
recovery_only	2.00	262	0	0	2,737	10.5	0%
optimization_only	2.00	258	0	0	2,998	9.22	0%
schema_only	2.00	274	0	0	2,550	5.73	0%
full	4.00	214	0	0	5,946	15.0	0%

given in the table. The policy agrees with the oracle on all 5547/5547 graded cases, with 0 gate errors. These figures were measured after the two contract-layer fixes described in lesson L6 (Section VII-F), which an earlier pass of the same corpus made necessary. They therefore describe the fixed system, and the corpus that exposed the gaps is now a committed regression test for this claim.

VII. PRODUCTION-HARDENING LESSONS

The central finding of Section VI-E has a less statistical counterpart. The system had several defects that its own test suite could not see, for the same reason that the mock evaluation flatters the headline numbers: the path the tests exercised was not the path production takes. Each lesson below was found through an audit or an experiment rather than through a failing test, and each is documented in the accompanying artifact.

A. L1: The Shared Resource Nobody Declared Concurrency-Unsafe

Although the base design describes execution as deterministic and auditable, our code for creating a versioned partition read the current maximum version and then dropped, created and registered a table in separate statements with no lock held, so that two callers working on the same partition at once could arrive at the same next version. The symptom was an intermittent failure in continuous integration. Rather than rerun the job until it passed, we traced it and found that a replay issued by the recovery agent had triggered a DAG that an unrelated test was using at the same moment. The read-modify-write is now protected by a database advisory lock. Lesson: once an agent is allowed to start real work, any resource it shares with other actors needs concurrency control, and a single-actor test suite will not reveal the problem.

B. L2: An Action Could Be Lost From the Audit Trail

In the first version the audit row was written after execution. A crash in that window, whether an out-of-memory

TABLE VII
Closed-Form Break-Even Points: The Assumed Median Human Latency (s) and Static Provisioning Level (Units) Below Which Each Configuration No Longer Beats Its Comparator

Config	MTTR (s)	Human break-even (s)	Cost reduction (%)	Static break-even
autoscale	52.4	49.9	47.6	3.00
full	235	224	55.7	2.15
monitor_only	359	342	−676	–
optimization_only	267	254	51.4	2.60
recovery_only	220	209	−676	–
rule_based	30.0	28.5	0	–
schema_only	315	300	−484	–

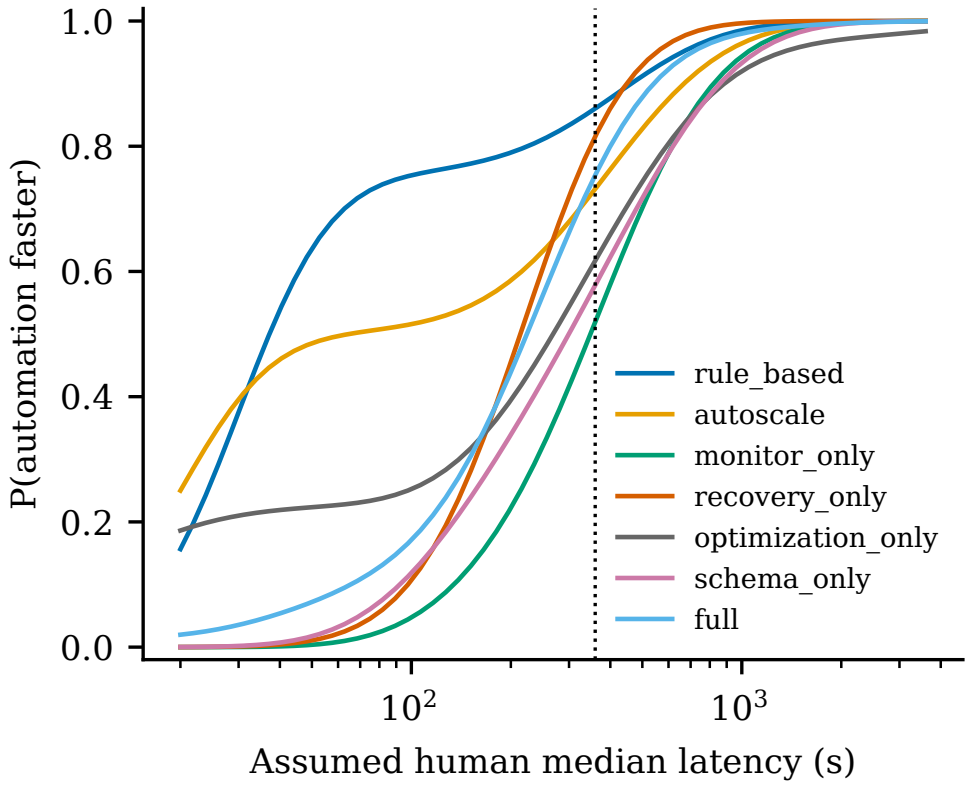


Fig. 7. Probability that automation recovers faster than the simulated human, as a function of the assumed human median latency.

kill or a lost database connection, could therefore let an action take effect without any record of it, and because the control loop catches per-tick exceptions the loss would probably have gone unnoticed. Since the system's central promise is that every action is both gated and audited, a single unaudited action would be enough to break it. The intent row, which already contains the policy verdict, is now committed before execution and completed with the outcome afterwards. Lesson: the record of what an agent intends to do should be durable before the agent does it.

C. L3: The Detector That Had No Callers

Only the fault injector wrote to the table that both the recovery-time statistics and decision scoring read from. The monitoring agent updated the detection time on a row that already existed but never inserted one. The result was a complete, deterministic anomaly detector, with unit tests, that nothing called. Without the injector, a real incident that the agent detected would have reached the audit table but never become an incident record, and production would have been unable to reproduce the experiment's own metrics. We noticed this while planning a later feature, by reading the code; no test failed. Lesson: a component can pass all of its own tests and still be disconnected, so follow the data from production input all the way to the reported metric.

D. L4: Two Taxonomies That Never Met

Decision correctness is scored against a set of accepted mitigations (Eq. (5)), and that set is keyed by the name

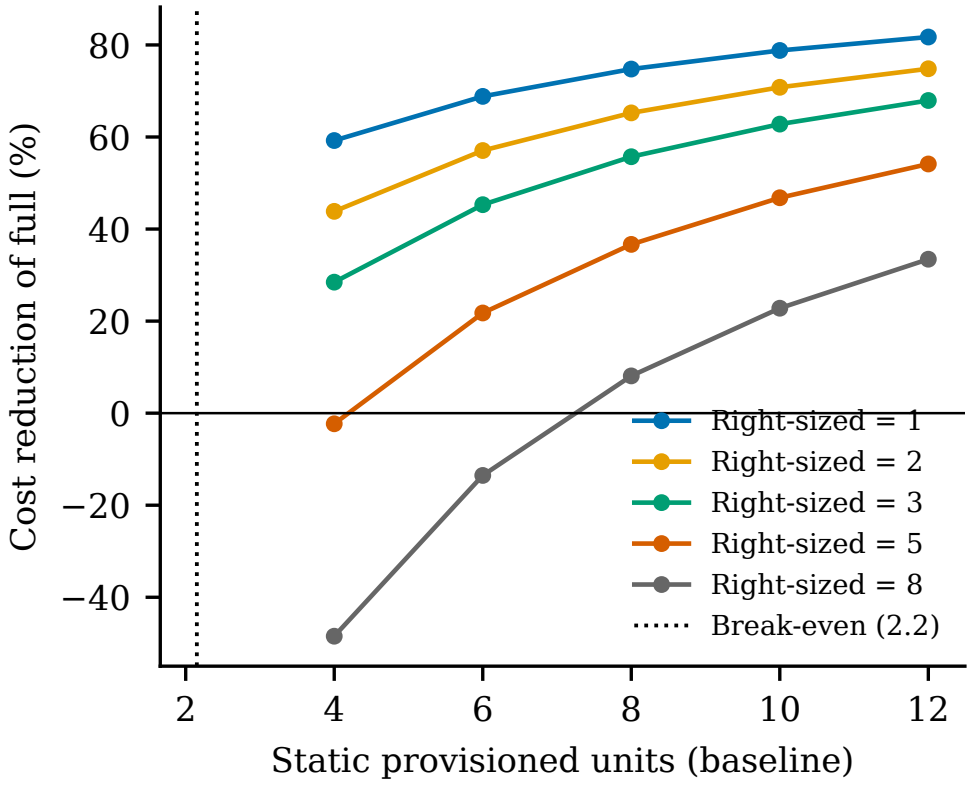


Fig. 8. Cost reduction of full relative to baseline for static provisioning levels from 4 to 12 units, one curve per right-sized level; the dotted line marks the break-even point.

TABLE VIII
Adversarial Corpus Graded Against the Independent Specification Oracle, by Category

Category	Cases	Containment [Wilson 95%]	Over-block	Oracle agreement	Fail-open
boundary_grid	4320	3056/3056 [0.9987, 1.0000]	0/1264 [0.0000, 0.0030]	4320/4320 [0.9991, 1.0000]	0
defense_in_depth	133	133/133 [0.9719, 1.0000]	–	133/133 [0.9719, 1.0000]	0
fuzz	1000	606/606 [0.9937, 1.0000]	0/394 [0.0000, 0.0097]	1000/1000 [0.9962, 1.0000]	0
injection	28	28/28 [0.8794, 1.0000]	–	28/28 [0.8794, 1.0000]	0
param_extremes	66	44/44 [0.9197, 1.0000]	0/22 [0.0000, 0.1487]	66/66 [0.9450, 1.0000]	0
Overall	5547	3867/3867 [0.9990, 1.0000]	0/1680 [0.0000, 0.0023]	5547/5547 [0.9993, 1.0000]	0

of the injected scenario. Live incidents use different fault-type names. Applying the set to live incidents would have marked every real incident "incorrect by construction." A mapping from live fault types onto the same accepted-mitigation set was added before the metric was exposed outside the experiment harness. Lesson: a metric defined in the experiment's own vocabulary cannot move into production without an explicit mapping.

E. L5: Hermeticity by Enumeration Does Not Scale

One class of unit-test failure kept recurring: a test that did not mock one module's database handle turned a millisecond test into a thirty-second connection-pool timeout. We patched it one test at a time, four times over, before dealing with the cause. All such calls now go through a single connection-pool accessor, which the test setup patches to fail at once. When we removed one essential mock to check the guard, six tests failed within seconds and each one named the cause. Lesson: a bug that has been fixed four times in four places is a sign that the mechanism producing it needs to change.

F. L6: What the Adversarial Corpus Found

When the generated corpus was graded against the independent oracle (the numbers are in Section VI), three findings emerged. The first was a pair of gaps in the contract layer. Because the cost policy compared a scaling target with the budget without checking a lower bound, a target of zero workers, which stops ingestion altogether, passed as within budget, and a negative "scale-down" passed as well. Separately, a non-numeric target made the gate raise an exception before the audit row had been written, so a proposal that was in effect rejected could leave no trace. Both gaps have been closed in the contract

layer, which now requires scaling targets to be integers no smaller than one and sends any violation down the normal invalid-output path.

The second finding concerned the policy engine itself. Run under a newer release than the one pinned for this project (Table I), some floating-point budget comparisons came out as within budget when they were not, while the pinned release handled them correctly. We have neither traced the root cause nor reported it upstream. Whatever the cause turns out to be, the pin is needed, and the corpus now also serves as an integration test that should catch any later upgrade that alters these semantics.

Third, once the contract fixes were in, the policy agreed with the independent specification on all graded cases, so containment is now a measured quantity with a confidence interval rather than an assertion inherited from the code under test. Lesson: a policy that its authors consider safe should be regarded as untested until an oracle written independently of it has tried to break it and failed.

G. L7: The Pilot Found a Retired Model

Before the live campaign, an eight-run pilot at the real paper timings measured cost and wall time. Within minutes it found that the provider had retired the small model configured for the monitoring agent (HTTP 410, end of life). Because monitoring is the detector in every agent configuration, the full campaign would otherwise have run with a detector degraded to a no-op, and every downstream number would have measured the provider's retirement of the model rather than the system. The supervisor has a guard against streaks of degraded calls, so it would have caught the problem eventually, but only after several runs had been wasted. The pilot caught it on the first call. Lesson: in a long experiment that costs money, the cheapest validity check is a short run with the real settings. A model that a provider still lists may no longer answer.

H. L8: The Simulated Engineer Rewarded Repeated Escalation

After the campaign we found a flaw in the on-call model that favours any configuration escalating more than once. The simulator gave each escalation row its own log-normal delay and closed the fault when the fastest one finished. An agent that escalated on several ticks of the same fault thus drew several delays and kept the shortest.

Table IX shows the effect. We count a run as human-resolved when its MTTR is at least 60 s; the chance that even the fastest of 6 draws falls below that is about 0.001. The human resolved 73 of the 80 full runs, and those runs escalated a median of 6 times. Their median MTTR, 241 s, is much closer to the 195 s expected from the fastest of 6 draws than to the 360 s of a single draw. recovery_only shows the same pattern (220 s), whereas monitor_only, which escalates once, stays at 359 s, close to baseline. We conclude that most of the MTTR reduction reported for full and recovery_only in Section VI was produced by the simulator and not by the agents, and that some of the increase in manual interventions has the same origin, since each re-escalation was recorded as a further row.

TABLE IX
Human-Resolved Runs ("Human"), Their Median Escalations and MTTR (s), and the Median Fastest of That Many Simulated-Human Draws (s)

Config	n	Human	Escal.	MTTR	Fastest
baseline	80	80	1	378	360
rule_based	80	20	1	433	360
autoscale	80	40	1	380	360
monitor_only	40	40	1	359	360
recovery_only	40	40	6	220	195
optimization_only	40	31	2	347	274
schema_only	40	40	1	315	360
full	80	73	6	241	195

The simulator now pages the engineer once per incident, starting at the first escalation, with a single delay per fault that paired runs share (D-106 in the repository). A pre-registered re-run with the corrected simulator is planned, but none of its numbers appear here. Lesson: because a simulated human is part of the measuring instrument, it has to be checked as carefully as the system under test, and in particular its output should not depend on how often the system asks it for help.

VIII. DISCUSSION

A. Implications for Evaluating Governed Agents

Taken together with the lessons of Section VII, the difference between mock and live runs (Section VI-E) suggests four practices for evaluating governed-agent systems. The first is to state whether the model in the loop was real or mocked and, where both numbers are available, to report both, since a figure obtained with a mock mainly characterizes the mock. The second is to include inexpensive non-LLM baselines. Here, two idealized ones beat the full agent system on recovery time and manual interventions, and a comparison with static orchestration alone would have hidden that. Third, grade safety claims against an oracle that shares no code with the mechanism being tested. Ours found two contract-layer gaps that the system's own tests had missed. Fourth, before a long campaign, pay for a short pilot at the real settings. Ours hit a retired model on its very first call.

B. Deployment Implications

For deployment, the most relevant decision-quality figure is 0.42 (Table III), not the headline MTTR: at these settings full executes an accepted mitigation in fewer than half of its runs. Together with the rise in manual interventions, this campaign gives no support for running this class of system unattended and fully autonomously against real infrastructure. The results instead favor the graduated trust ladder of Section IV as the safer production default: shadow, approval, and autonomous modes, with a durable kill switch and a per-target action cap. Nor do we think the full autonomy used as the research

benchmark's default should be read as a recommendation for deployment. For now, the trust core is an architectural response to a question that our measurements raise; we have not yet measured whether it works.

C. The Manual-Intervention Mechanism Is Measured, Not Explained

Section VI shows that full and recovery_only raise manual interventions compared with baseline, which contradicts the base paper. The measurement is straightforward: the metric counts rows in the telemetry.manual_ interventions table, and every configuration writes to that table. Why the count rises is a separate question, and we have not answered it. Possible explanations include a fault that remains open and is escalated again on successive ticks, and repeated denials that eventually end in a hand-off to the human, and the two may well occur together. Separating them would require a per-tick trace of proposal outcomes over the lifetime of a fault, which the current telemetry does not record. Section VII-H does settle part of the question: the median human-resolved full run escalated 6 times (Table IX), so repeated escalation on an open fault contributes at least some of the increase. Beyond that we leave the mechanism open rather than infer it from the totals. Readers should not assume it resolves in the system's favor.

D. Limitations and Threats to Validity

1) Threats to Validity: We see nine threats. Simulated human. All comparisons with baseline depend on the log-normal on-call model. Section VI-G gives the break-even point; it does not argue that the model is right. Per-escalation human draws. The simulator drew one delay per escalation and kept the fastest, so configurations that escalate repeatedly get shorter MTTRs. Section VII-H attributes most of the MTTR gain of full and recovery_only to this effect. Modelled baselines. In rule_based and autoscale, covered faults are resolved after fixed latencies that we assumed (Section V). Their lead over the agents comes from those assumptions and is not a measurement of any deployed rule engine or autoscaler. Synthetic workload. Our data are a downscaled synthetic table with injected faults, not production traces, and real workloads may behave differently. One live model family. The live arm used one reasoning model and one faster model from a single provider at temperature zero, so the decision-quality findings (Section VI) are properties of that pairing and may not generalize. Unequal replicate counts. baseline, rule_based, autoscale, and full each have 80 runs, whereas the four single-agent ablations have half as many. We made this trade-off for cost reasons before the campaign started (Section V), and it leaves the ablations with wider confidence intervals than the headline comparisons. Nominal seed matching. Because the per-run seed depends on the configuration name (Section V), no two configurations share fault or human-latency randomness at a matched (scenario, replicate) index. We therefore used the unpaired rank-sum test as the primary test, and the retained signed-rank statistic should not be read as evidence from a paired design. Detection-clock asymmetry. Non-agent configurations detect at injection by construction, whereas agent configurations start the MTTR clock only when the monitoring agent flags the fault (Section III). The agents' recovery times therefore omit a detection interval that the non-agent figures include, which favors the agents in every MTTR comparison. Cost measurement. The metered term of Eq. (4) is integrated between two resource samples per run, which spans the 300 s control loop for agent configurations but only a few seconds for non-agent ones. In addition, the Airflow pool size and the streaming-worker target carry over from one run to the next because the campaign never reset them. Cost comparisons therefore rest mainly on the provisioning term, an assumption whose break-even point Section VI-G reports, and the metered part should not be read as a like-for-like measurement.

2) Scope of the Testbed: The testbed is a single machine. The resource-contention scenario applies real CPU stress, which is why runs cannot be parallelized and why the size of the campaign is bounded. The policy engine is pinned to one release (Table I), and lesson L6 (Section VII-F) shows that a newer release from the same vendor would have changed the adversarial result had the version not been pinned. We disclose this as a property of our evaluation, not as a defect we investigated in the policy engine. Only four fault types were injected, whereas a real pipeline's incident taxonomy is larger, and lesson L4 (Section VII-D) shows that mapping an injected fault type onto a real incident type is not automatic.

3) Evidence Gaps: Two components described in Section IV are implemented but not evaluated, and a third gap lies within a contribution that is otherwise evaluated. Cross-model generalization. The harness that could test the base paper's claim of model independence is unit-tested, but its live sweep against real provider models has not been run, so nothing here is evidence for or against that claim. The trust core. Graduated autonomy, the kill switch, and the blast-radius cap are implemented and unit-tested, but every configuration ran in one fixed mode, so no mode transition or kill-switch-to-halt latency was measured. Cost. Internally the cost model of Eq. (4) splits compute from storage, but the per-run data we committed store only the total, and the metered term carries the confound described above.

None of these gaps needs much more than a well-defined piece of additional work. For the cross-model question, the sweep would be run on the same scenarios against at least two live providers, and the trust core could be tested with a drill that engages the kill switch and a mode transition under load while timing both. The cost problem is of a different kind, a flaw in our measurement rather than missing evidence; it would be removed by a re-run in which the pool size and worker target are reset before every run, resources are sampled at a fixed rate over the same window for all configurations, and the component breakdown of the ledger is kept.

4) What Would Change Our Conclusion: The test we would most like to see is a repetition of the live-versus-mock comparison of Section VI-E with a different provider. A gap of the same order of magnitude would strengthen the central claim considerably; a much smaller one would point to a property of this particular model pairing rather than of mock evaluation as such, and we would report that result in the same way.

IX. CONCLUSION AND FUTURE WORK

This paper reported an implementation and evaluation of a policy-gated multi-agent design for governing cloud pipelines, using a live remote language model, explicit non-agent baselines, an adversarial corpus graded by an independent oracle and closed-form sensitivity analyses. The design is only partly supported by the evidence. Against static orchestration it shortens recovery, though less than the original report claimed, and most of that reduction turned out to come from the simulated engineer rather than from the agents; it lowers cost by more than claimed, but almost entirely through an assumed provisioning gap; and it increases manual interventions, while two idealized non-LLM comparators recover faster still. In our judgement the most important result is about the evaluation rather than the design. Run along the same code path with the same configuration, a deterministic mock and a live model led to quite different conclusions, with recovery times about three orders of magnitude apart and a perfect correct-mitigation score under the mock that the live model did not come close to. For systems of this kind, then, the evaluation path behind a number matters at least as much as the number itself. We have deliberately kept the practical conclusion narrow and draw nothing from it about LLM agents in operations generally; what it does imply is that a governance claim should be accompanied by an account of how it was evaluated, and that mock and live numbers should be treated as separate findings until both have been measured.

Future work falls into two parts. One is to collect the evidence this campaign lacks: a live cross-model sweep to test model independence, a test of the trust core's mode transitions and kill-switch latency under load, a pre-registered re-run with the corrected simulator, per-run state resets and a fixed cost window, and deployed controllers and real production traces in place of the two modelled baselines (Section VIII). The other is design work: replacing the static agent-priority bid with an explicit negotiation protocol, enabling the bounded-adaptation mechanism that is currently disabled to preserve determinism, and extending the evaluation to multi-cloud deployments and to formal verification of the policy layer.

DATA AND CODE AVAILABILITY

The implementation, the Rego policies, the per-run campaign data for all 560 runs, the adversarial-corpus results, and the scripts that regenerate every table, figure, and reported number in this paper are available at https://github.com/bodapatisaikrishna/agentic-cloud-pipeline-governance.

ACKNOWLEDGMENT

Generative AI tools (Claude, Anthropic) were used to assist with software development, analysis scripts, and editing of the manuscript text. The authors reviewed all AI-assisted output, checked every reported result against the committed campaign data, and take full responsibility for the content of this paper.

References

[1] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," J. Syst. Softw., vol. 207, p. 111855, 2024, doi: 10.1016/j.jss.2023.111855.

[2] Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in Proc. 19th Eur. Conf. Comput. Syst. (EuroSys), 2024, pp. 674–688, doi: 10.1145/3627703.3629553.

[3] Z. Wang et al., "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2024, pp. 4966–4974, doi: 10.1145/3627673.3680016.

[4] Y. Luo et al., "OpsAgent: An evolving multi-agent system for incident management in microservices," in Proc. 41st IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE), 2026, pp. 3896–3907, doi: 10.1145/3832783.3834479.

[5] A. M. Kirubakaran et al., "Governing cloud data pipelines with agentic AI," Int. J. Comput. Sci. Trends Technol. (IJCST), vol. 13, no. 6, Dec. 2025, doi: 10.5281/zenodo.18048728; also arXiv:2512.23737.

[6] X. Liu, T. Chen, W. Li, X. Hu, and H. Wei, "The sim-to-real gap of foundation model agents: A unified MDP perspective," in Proc. 32nd ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD), Blue Sky Ideas Track, 2026, pp. 13 192–13 198, doi: 10.1145/3770855.3818660.

[7] M. Zhu et al., "RealUserSim: Bridging the reality gap in agent benchmarking via grounded user simulation," arXiv:2605.20204, 2026, doi: 10.48550/arXiv.2605.20204.

[8] A. Gupta, "ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions," arXiv:2601.06112, 2026, doi: 10.48550/arXiv.2601.06112.

[9] K. Rzadca et al., "Autopilot: Workload autoscaling at Google," in Proc. 15th Eur. Conf. Comput. Syst. (EuroSys), 2020, pp. 1–16, doi: 10.1145/3342195.3387524.

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track, 2024, doi: 10.48550/arXiv.2406.13352.

[11] H. Zhang et al., "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in Proc. 13th Int. Conf. Learn. Represent. (ICLR), 2025, doi: 10.48550/arXiv.2410.02644.

[12] A. Joshi, T. Finin, K. P. Joshi, and L. Kagal, "Deontic policies for runtime governance of agentic AI systems," in Proc. IEEE Int. Conf. Web Services (ICWS), 2026, pp. 1255–1264, doi: 10.1109/ICWS72778.2026.00159.

[13] A. C. Sanna, "Cross-LLM generalization of behavioral backdoor detection in AI agent supply chains," arXiv:2511.19874, 2025, doi: 10.48550/arXiv.2511.19874.

[14] L. Zhang, Y.-Y. Chu, and R. Krishnan, "Toward agentic governance: What shapes LLM-agent intervention in public forums?" arXiv:2606.00603, 2026, doi: 10.48550/arXiv.2606.00603.

[15] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. 11th Int. Conf. Learn. Represent. (ICLR), 2023, doi: 10.48550/arXiv.2210.03629.

[16] F. F. Xu et al., "TheAgentCompany: Benchmarking LLM agents on consequential real world tasks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 38, 2025, pp. 10 291–10 321, doi: 10.52202/085713-0315.

[17] J. Rystrøm, C. Schmitz, K. Korgul, J. Batzner, and C. Russell, "Agent benchmarks fail public sector requirements," arXiv:2601.20617, 2026, doi: 10.48550/arXiv.2601.20617.
[18] D. Hadfield-Menell, A. Dragan, P. Abbeel, and S. Russell, "The off-switch game," in Proc. 26th Int. Joint Conf. Artif. Intell. (IJCAI), 2017, pp. 220–227, doi: 10.24963/ijcai.2017/32.
[19] H. Zheng, Q. Dong, R. K. Depena, J. D. Bhatia, F. Xiao, and P. Xu, "Separating capability from permission: A governance framework for agentic AI autonomy levels," arXiv:2607.23438, 2026, doi: 10.48550/arXiv.2607.23438.
[20] Cloud Native Computing Foundation, "Open Policy Agent: policy-based control for cloud native environments," https://www.openpolicyagent.org, 2016, accessed 2026-09-21.
[21] K. Zhang, Y. Peng, K. Jiang, and Y. Zhang, "ActGov: Governing LLM agent actions via policy-constrained validation," arXiv:2609.24446, 2026, doi: 10.48550/arXiv.2609.24446.
[22] Y. Chen et al., "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," arXiv:2501.06706, 2025, doi: 10.48550/arXiv.2501.06706.
[23] J. Barnes, "OpenSec: Measuring incident response agent calibration under adversarial evidence," arXiv:2601.21083, 2026, doi: 10.48550/arXiv.2601.21083.
[24] A. Basiri et al., "Chaos engineering," IEEE Software, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/MS.2016.60.
[25] B. Efron, "Bootstrap methods: Another look at the jackknife," Ann. Statist., vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.
[26] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," Ann. Math. Statist., vol. 18, no. 1, pp. 50–60, 1947, doi: 10.1214/aoms/1177730491.
[27] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bull., vol. 1, no. 6, p. 80, 1945, doi: 10.2307/3001968.
[28] S. Holm, "A simple sequentially rejective multiple test procedure," Scand. J. Statist., vol. 6, no. 2, pp. 65–70, 1979.
[29] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," Psychol. Bull., vol. 114, no. 3, pp. 494–509, 1993, doi: 10.1037/0033-2909.114.3.494.
[30] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. Amer. Statist. Assoc., vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.